

Security+ 2.3 · SQL Injection

Mestre CyberSec · Aula 2.3 — SQL Injection (5:09) · [Assistir Vídeo no YouTube](#) ↗ · Treino Interativo: edusouzamarques.github.io/cybersec

Resumo Executivo da Aula: Injetar SQL num campo de entrada: 'OR 1=1--' pula o login; DROP TABLE apaga a tabela; UNION SELECT extrai dado. Defesa: PREPARED STATEMENT (parameterized query), input validation, WAF, menor privilégio no banco. Nunca concatenar string pra query.

💡 Highlights que a prova CompTIA cobra

- SQLi = input injetado no query sem sanitizar
- Defesa nº1: prepared statement / parameterized query
- Defesa nº2: menor privilégio (o banco do user não tem Drop Table)

🎯 Dicas de Prova & Cenários Típicos da Banca

- 'input concatenated into a SQL query' = SQL injection
- 'parameterized queries prevent' = defesa do SQLi (prepared statement)

Por que este aprofundamento é cobrado na prova?: O vídeo resume os tópicos práticos, mas o exame CompTIA SY0-701 cobra discernimento técnico rigoroso sobre as tecnologias e protocolos específicos desta seção:

• Injeção de SQL (SQLi)

Ataque de injeção em que entrada não confiável é concatenada em uma query SQL, permitindo ao invasor alterar a lógica da consulta e acessar ou destruir dados do banco.

⚠ **Armadilha CompTIA:** *Trigger phrase 'input concatenated into a SQL query' = SQLi. Use '--' or ' OR 1=1' as classic textbook payload examples.*

• Instrução Preparada (Query Parametrizada) (N/A)

Técnica de codificação em que o código SQL e a entrada do usuário são enviados ao SGBD de forma separada, tratando-a apenas como dado, sem alterar a estrutura da query.

⚠ **Armadilha CompTIA:** *Trigger phrase 'parameterized queries prevent' = the #1 defense for SQL injection on the exam. Prefer this over escaping or WAF when both are options.*

• Validação de Entrada (N/A)

Verificação no servidor da entrada do usuário contra formato, tipo ou faixa esperada (ex.: allowlist alfanumérica) antes do processamento, reduzindo risco de injeção.

⚠ **Armadilha CompTIA:** *Distinguish from sanitization: validation rejects bad input, sanitization cleans it. For SQLi, parameterized query is still preferred even when validation exists.*

• Firewall de Aplicação Web (WAF)

Firewall de camada 7 que inspeciona tráfego HTTP/S para uma aplicação web e bloqueia payloads maliciosos conhecidos como SQLi, XSS e injeção de comando.

⚠ **Armadilha CompTIA:** *WAF is compensating control, NOT a substitute for parameterized queries. On the exam, the best fix for SQLi is always code-level, not network-level.*

• Princípio do Menor Privilégio (PoLP)

Conceder à conta de banco apenas as permissões mínimas necessárias (ex.: somente SELECT/INSERT), impedindo que um SQLi bem-sucedido faça DROP de tabelas ou escale privilégios.

⚠ **Armadilha CompTIA:** *Trigger phrase 'database user cannot DROP TABLE' = least privilege mitigated impact. Pair with PoLP for defense-in-depth, not as primary SQLi fix.*

• Ataque UNION SELECT (N/A)

Payload de SQLi que usa o operador UNION para anexar um segundo SELECT, exfiltrando dados de outras tabelas do mesmo banco pela saída da query vulnerável.

⚠ **Armadilha CompTIA:** *If exam describes 'extracting data via UNION' or 'appended SELECT', it is SQLi, not XSS or command injection. Requires equal column count to succeed.*

• Comando DROP TABLE (N/A)

Comando DDL destrutivo do SQL frequentemente injetado via SQLi (ex.: '; DROP TABLE users--') para excluir tabelas inteiras, causando DoS ou perda de dados.

⚠ **Armadilha CompTIA:** *Trigger phrase 'database user should not have permission to DROP' = least privilege mitigation. Multi-statement execution must be enabled for this payload to work.*

• Vulnerabilidade de Concatenação de Strings (N/A)

Padrão de código inseguro que monta SQL juntando strings (ex.: 'SELECT * FROM users WHERE nome="' + entrada + "''), permitindo ao atacante sair do contexto de dado para o de código.

⚠ **Armadilha CompTIA:** *Any exam question showing query built via '+' or string concatenation is the hallmark vulnerable pattern — fix is parameterized query, not escaping.*

✓ Perguntas de Treino Comentadas (10 Questões de Cenários Reais)

Cenários autênticos alinhados aos objetivos do exame. O gabarito com análise completa dos distratores encontra-se ao final.

QUESTÃO 1 · CENÁRIO DE EXAME

A página de login de uma empresa de varejo autentica usuários montando esta consulta: `SELECT * FROM Users WHERE username = ' ' AND password = ' '`. Um testador de penetração insere `' OR 1=1--` no campo de usuário e entra como o primeiro usuário da tabela. Qual mudança aborda melhor a causa raiz dessa vulnerabilidade?

- A) Substituir SQL concatenado dinamicamente por instruções preparadas com parâmetros.
- B) Exigir que usuários escolham senhas mais longas.
- C) Ativar bloqueio de conta após três tentativas falhas.
- D) Forçar TLS 1.3 para todo o tráfego de autenticação.

QUESTÃO 2 · CENÁRIO DE EXAME

Durante uma avaliação de segurança, um testador descobre que uma página de busca de produtos pega o termo de busca e o anexa a uma consulta. Ao enviar um termo manipulado, o testador faz o banco de dados retornar nomes de usuário e hashes de senha na coluna de descrição do produto. Qual controle impede mais diretamente esse tipo de extração de dados?

- A) Implantar um firewall de aplicação web em modo somente monitoramento.
- B) Usar consultas parametrizadas para a operação de busca.
- C) Ocultar mensagens de erro do banco de dados dos usuários.
- D) Limitar os resultados da busca a dez registros por página.

QUESTÃO 3 · CENÁRIO DE EXAME

Uma aplicação web se conecta ao banco de dados com uma conta que possui direitos `db_owner`. Um atacante explorando a entrada de um formulário tenta executar uma consulta empilhada que apaga a tabela `Orders`. Qual controle do lado do banco reduziria mais o impacto se a aplicação for injetada com sucesso?

- A) Ativar registro detalhado de consultas SQL.
- B) Armazenar backups no mesmo servidor de banco de dados.
- C) Conceder à conta da aplicação apenas as permissões exigidas por suas funções.
- D) Adicionar um CAPTCHA ao formulário vulnerável.

QUESTÃO 4 · CENÁRIO DE EXAME

Uma revisão de código encontra o seguinte trecho Java: `String q = "SELECT * FROM accounts WHERE acct = '" + acct + "'";` Qual substituição remove melhor a falha de injeção SQL?

- A) Substituir aspas simples em `acct` por duas aspas simples antes da concatenação.
- B) Limitar `acct` a dez caracteres alfanuméricos.
- C) Mover a consulta para um procedimento armazenado que usa EXEC com `acct` concatenado.
- D) Usar um PreparedStatement com um placeholder `?` e vincular `acct` como parâmetro.

QUESTÃO 5 · CENÁRIO DE EXAME

Um engenheiro de segurança confirma que um firewall de aplicação web está bloqueando padrões conhecidos de injeção SQL, mas a aplicação ainda concatena entrada do usuário em consultas SQL. Qual controle adicional reduz mais diretamente o risco na camada da aplicação?

- A) Refatorar o acesso ao banco de dados para usar consultas parametrizadas.
- B) Segmentar o servidor web e o servidor de banco em VLANs diferentes.
- C) Ativar monitoramento de integridade de arquivos no servidor de banco.
- D) Exigir autenticação multifator para administradores do banco.

QUESTÃO 6 · CENÁRIO DE EXAME

Um desenvolvedor adiciona validação no lado do servidor para que um campo de ID de cliente contenha apenas dígitos antes de usá-lo em uma consulta ao banco. Qual afirmação descreve melhor o valor de segurança desse controle?

- A) Ele impede totalmente a injeção SQL e remove a necessidade de consultas parametrizadas.
- B) Ele reduz a superfície de ataque, mas deve ser combinado com consultas parametrizadas.
- C) Ele é eficaz apenas quando a validação JavaScript no lado do cliente também está ativada.
- D) Ele substitui a necessidade de privilégio mínimo na conta do banco.

QUESTÃO 7 · CENÁRIO DE EXAME

Um administrador de banco descobre que a conta de banco da aplicação web tem privilégios SELECT, INSERT, UPDATE, DELETE e DROP em todos os esquemas, embora a aplicação apenas leia de duas views e escreva em uma tabela. Qual ação reduz melhor o impacto potencial de uma conta de aplicação comprometida?

- A) Renomear a conta de banco da aplicação para um nome menos óbvio.
- B) Ativar TLS para todas as conexões de banco de dados.
- C) Revogar privilégios desnecessários e conceder acesso apenas a views e operações de tabela exigidas.
- D) Mover o banco de dados para um segmento de rede separado.

QUESTÃO 8 · CENÁRIO DE EXAME

Uma aplicação retorna erros brutos do banco de dados aos usuários, incluindo nomes de tabelas, colunas e detalhes de sintaxe SQL. Um atacante usa essas mensagens para refinar tentativas de injeção SQL. Qual ação reduz melhor essa divulgação de informações e ainda apoia a investigação de incidentes?

- A) Desativar completamente o registro de consultas do banco.
- B) Configurar o WAF apenas para registrar requisições sem bloquear.
- C) Aumentar a complexidade das senhas dos usuários da aplicação.
- D) Retornar mensagens de erro genéricas aos usuários e registrar erros detalhados no servidor.

QUESTÃO 9 · CENÁRIO DE EXAME

Uma aplicação legada não pode ser modificada imediatamente para usar consultas parametrizadas. O arquiteto de segurança precisa de um controle compensatório temporário na camada HTTP para bloquear requisições maliciosas de injeção SQL até que o código seja corrigido. Qual controle se encaixa melhor nesse requisito?

- A)** Um firewall de aplicação web com assinaturas de injeção SQL em modo de bloqueio.
- B)** Criptografia de dados em repouso para todas as tabelas da aplicação.
- C)** Controle de acesso baseado em funções para usuários finais da aplicação.
- D)** Rotação de credenciais do banco a cada 30 dias.

QUESTÃO 10 · CENÁRIO DE EXAME

Uma equipe de desenvolvimento move várias operações de banco para procedimentos armazenados para melhorar a segurança. Um revisor descobre que um procedimento monta uma consulta com `EXEC('SELECT * FROM Inventory WHERE Item = ''' + @Item + ''')`. Qual conclusão é mais precisa?

- A)** Procedimentos armazenados sempre eliminam injeção SQL independentemente de como são escritos.
- B)** O procedimento permanece vulnerável porque a entrada ainda é concatenada em SQL dinâmico.
- C)** Injeção SQL afeta apenas consultas ad hoc, não procedimentos armazenados.
- D)** O procedimento é seguro se a conta do banco tiver privilégio mínimo.

GABARITO COMENTADO COM ANÁLISE DE DISTRATORES

Questão 1: Alternativa A — Substituir SQL concatenado dinamicamente por instruções preparadas com parâmetros.

Instruções preparadas com parâmetros separam o código SQL dos dados, então a entrada ' OR 1=1-- é tratada como um nome de usuário literal, e não como SQL executável. Senhas mais longas, bloqueio e TLS são úteis, mas não corrigem a falha de injeção.

Questão 2: Alternativa B — Usar consultas parametrizadas para a operação de busca.

O ataque descrito é uma injeção SQL baseada em UNION SELECT. Consultas parametrizadas impedem que a entrada do usuário seja interpretada como sintaxe SQL, bloqueando o UNION SELECT injetado. Um WAF em modo monitoramento apenas observa; ocultar erros não impede extração; e limites de resultados podem reduzir volume, mas não corrigem a falha.

Questão 3: Alternativa C — Conceder à conta da aplicação apenas as permissões exigidas por suas funções.

O privilégio mínimo limita o que o SQL injetado pode fazer. Se a conta da aplicação não puder apagar tabelas nem acessar objetos não relacionados, uma injeção bem-sucedida causa menos dano. Logs, backups no mesmo servidor e CAPTCHA não removem permissões excessivas no banco.

Questão 4: Alternativa D — Usar um PreparedStatement com um placeholder ? e vincular acct como parâmetro.

Um PreparedStatement com vinculação de parâmetros trata acct como dado e impede que ele altere a sintaxe SQL. Escape manual de aspas e limites de tamanho são incompletos, e um procedimento armazenado que usa EXEC com concatenação ainda pode ser vulnerável.

Questão 5: Alternativa A — Refatorar o acesso ao banco de dados para usar consultas parametrizadas.

Consultas parametrizadas corrigem a causa raiz ao impedir que a entrada seja executada como SQL. Segmentação de rede, monitoramento de integridade de arquivos e MFA para DBAs são controles úteis, mas não eliminam diretamente a vulnerabilidade de injeção SQL na aplicação.

Questão 6: Alternativa B — Ele reduz a superfície de ataque, mas deve ser combinado com consultas parametrizadas.

A validação de entrada pode rejeitar muitas entradas maliciosas e reduzir a superfície de ataque, mas não é uma defesa completa contra SQLi por si só. Consultas parametrizadas ainda são necessárias, e o privilégio mínimo continua importante.

Questão 7: Alternativa C — Revogar privilégios desnecessários e conceder acesso apenas a views e operações de tabela exigidas.

Privilégio mínimo significa conceder apenas as permissões mínimas necessárias. Revogar direitos amplos e limitar o acesso aos objetos exigidos reduz o impacto de injeção SQL e outros comprometimentos. Renomear, TLS e segmentação não resolvem privilégios excessivos no banco.

Questão 8: Alternativa D — Retornar mensagens de erro genéricas aos usuários e registrar erros detalhados no servidor.

Erros genéricos para o usuário evitam revelar detalhes de esquema e consulta, enquanto o registro no lado do servidor preserva informações de diagnóstico para investigadores. Desativar logs prejudica a investigação; WAF apenas com registro não reduz a divulgação; e complexidade de senha não resolve vazamento de erros.

Questão 9: Alternativa A — Um firewall de aplicação web com assinaturas de injeção SQL em modo de bloqueio.

Um WAF com assinaturas de SQLi em modo de bloqueio pode fornecer detecção e prevenção temporárias na camada HTTP enquanto o código legado é corrigido. Criptografia em repouso, RBAC para usuários e rotação de credenciais não inspecionam nem bloqueiam cargas de injeção SQL.

Questão 10: Alternativa B — O procedimento permanece vulnerável porque a entrada ainda é concatenada em SQL dinâmico.

Procedimentos armazenados não são automaticamente seguros. Se eles constroem SQL dinâmico com entrada do usuário concatenada e EXEC, a injeção ainda é possível. Privilégio mínimo reduz o impacto, mas não corrige o procedimento vulnerável; injeção não se limita a consultas ad hoc.

